Multi-Layer Offloading (MLO) on AMD Versal with PYNQ

Alexander Klassen

Motivation

Transformer-scale models cannot be directly mapped to FINN's dataflow architecture. Mapping every layer to its own compute unit exceeds the available on-chip memory and compute, making it infeasible on FPGA.
Multi-Layer Offloading (MLO) collapses repeated subgraphs into a single looped subgraph, streaming weights from external memory. This trades throughput for the ability to fit these models.

However, until now, MLO had three major limitations:

1. **Dependence on HBM for external weight storage**
2. **Deployment limited to a custom V80 flow**
   - Developed as part of an initial V80 proof-of-concept
   - No reusable FINN deployment flow for other Versal devices
3. **No generic driver support**
   - Driver support was tailored to the original deployment
   - No integration with the standard FINN/PYNQ workflow

Key Ideas

1. Make MLO memory agnostic
MLO has two types of components that interact with external memory:
- Matrix-vector units (MVUs), which fetch their weights
- The Intermediate Frames block, which stores intermediate activations

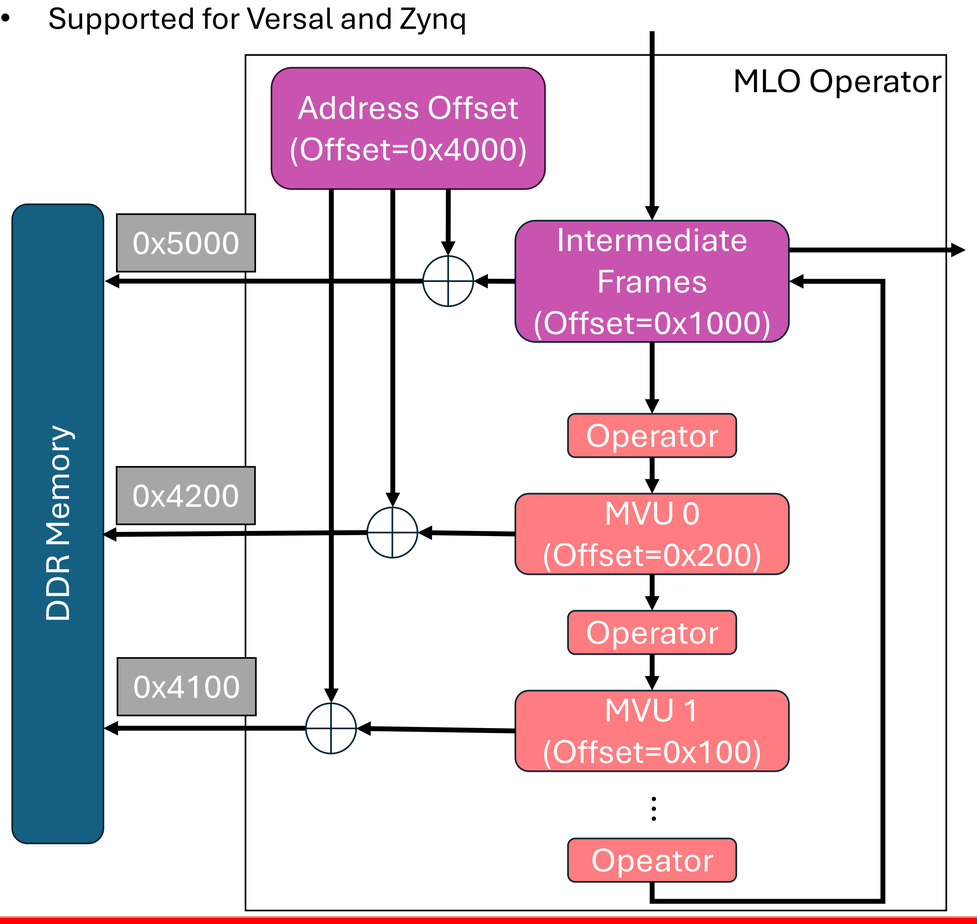
Previously, each component relied on a dedicated HBM resource. To support additional memory architectures, address offsets are introduced for external memory accesses.

2. Allow unified generation of a FINN Versal Shell
In FINN, shells are device-specific harnesses that connect accelerators to platform interfaces and support deployment using generated PYNQ drivers. While MLO has previously been demonstrated on a V80 platform, this relied on a custom deployment flow. The key idea is to extend the existing shell infrastructure and unify deployment across Versal and Zynq platforms.

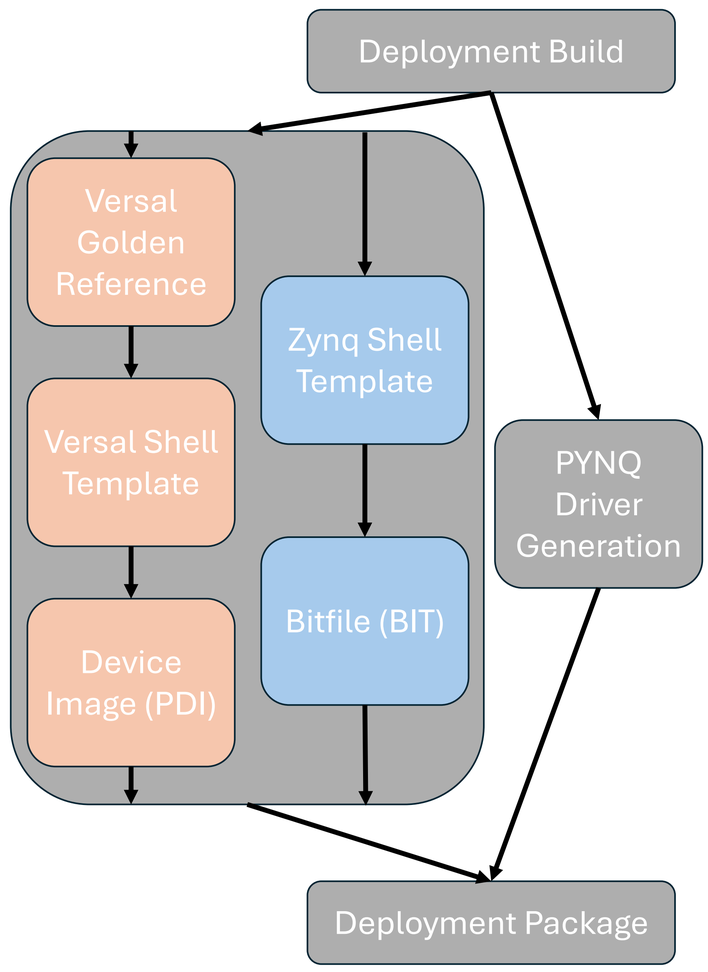
3. MLO Driver Support in PYNQ
MLO requires additional software support to configure and manage external weights. With the recent availability of PYNQ for Versal, the existing FINN PYNQ driver can be extended to support MLO deployments on both Versal and Zynq platforms.

Contribution

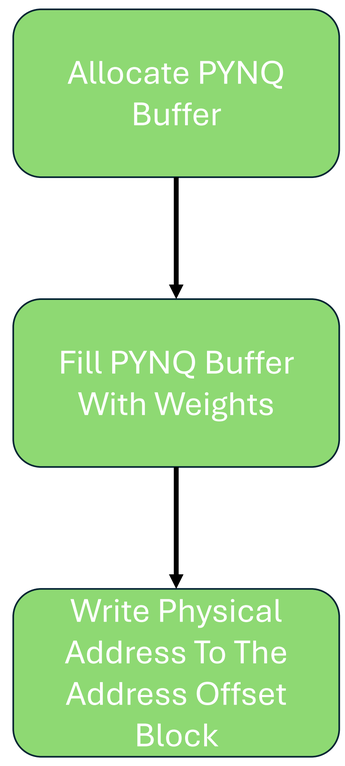
1. **Support DDR memory by allowing for address offsets in hardware**
   - Uses a single shared, contiguous memory region in DDR memory
   - Every component has a local address offset
   - A global address offset is set by a new hardware component
   - The total address is the sum of the local and global address offsets
   - Supported for Versal and Zynq



2. **Shell generation for Versal**
   - Requires a Versal PYNQ golden reference
     - The golden reference constrains the interfaces outside of the PL
       - E.g., the NoC and clock
   - Unifies the shell build infrastructure (Versal and Zynq)
     - Currently only used for VCK190 deployment



3. **MLO PYNQ driver support**
   - Usable for both Versal and Zynq
   - Allocates a single large PYNQ buffer for a whole MLO operator



Future Work

- Test support for different Versal boards
- Improve FINN's management of the golden references
- Evaluate the performance of the approach on accelerators

Discussion

- Support for DDR memory enables the use of a wider range of devices
- Shell generation for Versal allows for fast deployment of Versal devices in FINN
- The PYNQ driver is usable for both Versal and Zynq
  - For Versal, it must adhere to the golden reference constraints
  - PYNQ for Versal fixes the Vivado version to 2025.2

Contact: aklassen@amd.com